

# Mostek SC4-Hub

---

Demon C++ na sFoundation, który wystawia na TCP ten sam protokół tekstowy co firmware ClearCore. Serwer maszyny działa na serwach ClearPath-SC bez zmian w logice — wystarczy

`MACHINE_MODE=clearcore CLEARCORE_HOST=127.0.0.1`.

Przejechany na sprzęcie: pełny cykl programu `583912004711` na trzech serwach.

## Pliki

- `bridge/sc4hub_bridge.cpp` — mostek: serwer TCP, maszyna stanów, ruch osi, odczyt Global Stop, tryb `--identify` do mapowania osi, `--list`.
- `bridge/Makefile` — budowanie przeciw `vendor/teknib/lib`; cele `run`, `identify`.
- `bridge/machine.env` — mapowanie osi po numerach seryjnych, jednostki, limity.
- `server/app/machine.py` — stany RUNNING/PAUSED utrzymuje serwer.

## Poprawione błędy

**Stan cyklu gubiony na sprzęcie.** `poll_status()` chodzi co 0,2 s z pętli WebSocketu i nadpisywał stan tym, co zgłasza sterownik. Sterownik między ruchami zgłasza READY i nie wie o operacji PAUZA, więc panel pokazywał READY przez cały cykl, a PAUSED znikał — a `start()` rozpoznaje wznowienie właśnie po PAUSED. Teraz RUNNING i PAUSED ustawia serwer (jak w symulatorze), a `poll_status()` ich nie rusza; ALARM ze sterownika ma pierwszeństwo zawsze. Na symulatorze błąd nie występował — tam `poll_status()` nie jest używany.

**Rozdzielczość enkodera była zgadywana.** Wpisana na sztywno stała 6400 imp/obr dawała ruch 8× wolniejszy od zadanego (serwa mają **800 imp/obr**), przy pozornie poprawnym odczycie pozycji — bo błąd skracał się w przeliczeniu tam i z powrotem. Objawiał się dopiero przekroczeniem limitu czasu ruchu. Mostek odczytuje teraz `Info.PositioningResolution` z serwa i sprawdza, czy wszystkie osie mają tę samą wartość.

**Limit czasu ruchu liczony własnym wzorem.** Przy złym przeliczniku jednostek objawiał się jako „przekroczono czas ruchu” zamiast wskazać przyczynę. Teraz pochodzi z `MovePosnDurationMsec()` biblioteki, z zapasem  $1,5 \times + 3$  s.

**Mostka nie dało się zatrzymać.** `signal()` ustawia `SA_RESTART`, więc `accept()` był wznawiany i SIGTERM nic nie robił — trzeba było SIGKILL, po którym port szeregowy zostawał zajęty i kolejny start kończył się błędem inicjalizacji. Teraz `sigaction()` bez `SA_RESTART`.

**Alarm przy każdym rozłączeniu.** Rozłączenie klienta na postoju wpędzało mostek w `ALARM` i wymuszało `RESET`. Teraz alarm tylko wtedy, gdy maszyna była w ruchu.

## Świadome uproszczenia

- **Bazowanie jest udawane.** Homing ClearPath-SC konfiguruje się w ClearView (Windows). Gdy `HomingValid()` jest fałszywe, mostek robi **zerowanie programowe** (`AddToPosition`) — bieżąca pozycja staje się zerem. Wystarcza na stole, ale **nie jest bazowaniem** i nie nadaje się na maszynę z mechaniką.
- **Interpolacja XY jest przybliżona.** Prędkości osi dobrane tak, by obie skończyły jednocześnie; rampy przyspieszenia dają odchyłkę od prostej na końcach ruchu. Zmierzone: przejazd 22,36 mm przy 600 mm/min zajął 2,42 s wobec teoretycznych 2,24 s. Dla operacji `LINIA` wymaga weryfikacji pomiarowej.
- **Wrzeczono tylko śledzone** (`SPINDLE_OUTPUT=none`) — nic nie jest podłączone. Do wyjścia huba przełącza się przez `brake0/brake1`.
- **STOP w trakcie ruchu:** mostek nasłuchuje gniazda co 20 ms, więc STOP działa natychmiast, nie po zakończeniu ruchu. Odpowiedź na przerwany komendę ruchu jest **pomijana** — klient, który ją wysłał, jest w tym momencie anulowany po stronie serwera i nie czeka na odpowiedź.

## Konfiguracja

Wszystko w `bridge/machine.env`. Najważniejsze:

| Zmienna                                         | Wartość                           | Znaczenie                                                                                                         |
|-------------------------------------------------|-----------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>AXIS_X/Y/</code><br><code>Z_SERIAL</code> | 90406231 / 90406002 /<br>90404362 | mapowanie osi po S/N                                                                                              |
| <code>MM_PER_REV</code>                         | 5.0                               | skok śruby kulowej — dziś tylko wartość startowa, właściwe przełożenia przysyła serwer komendą <code>AXCFG</code> |
| <code>COUNTS_PER_REV</code>                     | (odczyt z serwa)                  | tylko do nadpisania                                                                                               |
| <code>MAX_RPM</code>                            | 400                               | twardy limit obrotów                                                                                              |
| <code>SPINDLE_OUTPUT</code>                     | none                              | none / <code>brake0</code> / <code>brake1</code>                                                                  |

## Uwagi

- Mostek **nie realizuje funkcji bezpieczeństwa** — czyta tylko Global Stop i odrzuca komendy ruchu bez zezwolenia, dokładnie jak zakładał firmware.
- Serwa włączane przy pierwszej komendzie ruchu, wyłączane przy STOP, rozłączeniu klienta i zamknięciu mostka.
- Bez podłączonego panelu (WebSocket) serwer nie odpytuje statusu, więc stan zostaje `INIT` i `START` odmówi. To zachowanie istniejącego kodu serwera.
- Nieprzetestowane: STOP w trakcie ruchu, utrata zezwolenia w trakcie ruchu, zachowanie przy wciśniętym E-stopie.